

→ LALR(1) :-

LALR(1) stands for Look-Ahead LR(1).

- L → Scan input from left to right
- A → look-Ahead
- LR → Rightmost derivation in reverse
- 1 → One lookahead symbol

main idea:

LALR(1) is constructed from CLR(1) states by merging states that have the same LR(0) core.

→ Why LALR(1) is needed?

CLR(1) is powerful, but it can create many states.

for example

Suppose CLR(1) has:

I_5 :

$[A \rightarrow \alpha \cdot, a]$

I_{10} :

$[A \rightarrow \alpha \cdot, b]$

Their LR(0) core is the same:

$$A \rightarrow \alpha.$$

Only their lookaheads differ.

LALR(1) can merge them:

$$[A \rightarrow \alpha., a/b]$$

Thus the number of states decreases.

→ Rule of LALR(1):~

Suppose two CLR(1) states are:

I_5 :

$$[A \rightarrow \alpha. \beta, a]$$

$$[B \rightarrow \gamma., c]$$

and

I_{10} :

$$[A \rightarrow \alpha. \beta, b]$$

$$[B \rightarrow \gamma., d]$$

Remove lookaheads :

I_s core :

$A \rightarrow \alpha \cdot \beta$

$B \rightarrow \gamma \cdot$

I_{10} core :

$A \rightarrow \alpha \cdot \beta$

$B \rightarrow \gamma \cdot$

The core are identical :

$\therefore$ merge them :

I_s / I_{10} :

$[A \rightarrow \alpha \cdot \beta, a/b]$

$[B \rightarrow \gamma \cdot, c/d]$

→ LALR Construction Steps :

CLR(1)



Find same LR(0) core



Merge States



LALR(1)

Eg:

Same example because we have to find CLR (1) parsing table.

$S \rightarrow cc$

$C \rightarrow cc$

$C \rightarrow d$

⇒

Canonical collection:

I_0 :

$[S' \rightarrow \cdot S, \$]$

$[S \rightarrow \cdot cc, \$]$

$[C \rightarrow \cdot cc, c/d]$

$[C \rightarrow \cdot d, c/d]$

I_1 :

$[S' \rightarrow S, \$]$

I_2 :

$[S \rightarrow Cc, \$]$

$[C \rightarrow \cdot cc, \$]$

$[C \rightarrow \cdot d, \$]$

I_3 :

$[C \rightarrow cC, c/d]$

$[C \rightarrow \cdot cc, c/d]$

$[C \rightarrow \cdot d, c/d]$

$I_4 :$

$[C \rightarrow d \cdot, c/d]$

$I_5 :$

$[S \rightarrow cc \cdot, \$]$

$I_6 :$

$[C \rightarrow c \cdot c, \$]$

$[C \rightarrow \cdot cc, \$]$

$[C \rightarrow d \cdot, \$]$

$I_7 :$

$[C \rightarrow d \cdot, \$]$

$I_8 :$

$[C \rightarrow cc \cdot, c/d]$

$I_9 :$

$[C \rightarrow cc \cdot, \$]$

Now,

here,

I_3 and I_6 has identical LR(0) case.

$\therefore$ we merge them.

$I_3 :$

$[C \rightarrow c \cdot c, c/d/\$]$

$[C \rightarrow \cdot c c, c/d/\$]$

$[C \rightarrow \cdot d, c/d/\$]$

Similarly I_4 and $I_7 :$

$I_4 :$

$[C \rightarrow d \cdot, c/d/\$]$

Similarly I_8 and $I_9 :$

$I_8 :$

$[C \rightarrow c c \cdot, c/d/\$]$

Date			
------	--	--	--

Hence this is LALR(1) parsing table
without conflict

let :

I 10'

$$[A \rightarrow d., c]$$
$$[B \rightarrow d., q]$$

⇒ This means:

for I_5 :

$[A \rightarrow d \cdot, a]$

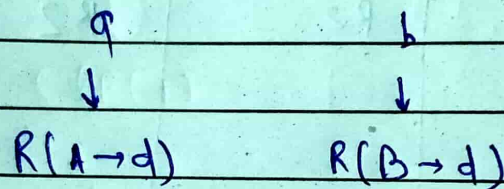
$[B \rightarrow d \cdot, c]$

⊙

· if next symbol is $a \rightarrow$ reduce $A \rightarrow d$

· if next symbol is $c \rightarrow$ reduce $B \rightarrow d$

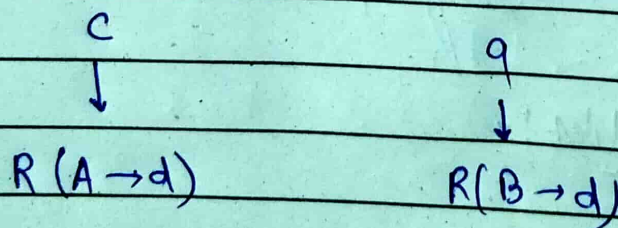
So I_5 has no conflict:



for I_{10} :

$[A \rightarrow d \cdot, c]$

$[B \rightarrow d \cdot, a]$



So I_{10} has no conflict

⊙ CLR(1) states are valid.

Now look at LR(0) core

They are identical

I_5 and I_{10}

$\therefore$ Merge them

I_5 / I_{10} :

$[A \rightarrow d \cdot, q/c]$
 $[B \rightarrow d \cdot, q/c]$

Now check lookahead :

q
├ $\rightarrow$ Reduce $A \rightarrow d$
└ $\rightarrow$ Reduce $B \rightarrow d$

$\therefore$ q has :

Reduce-Reduce Conflict

Same thing for c also.

→ Classification of LR family:

This is about grammar class/power.

LR(0)



SLR(1)



LALR(1)



CLR(1)

In term of capability :

$LR(0) \subseteq SLR(1) \subseteq LALR(1) \subseteq CLR(1)$

Akash